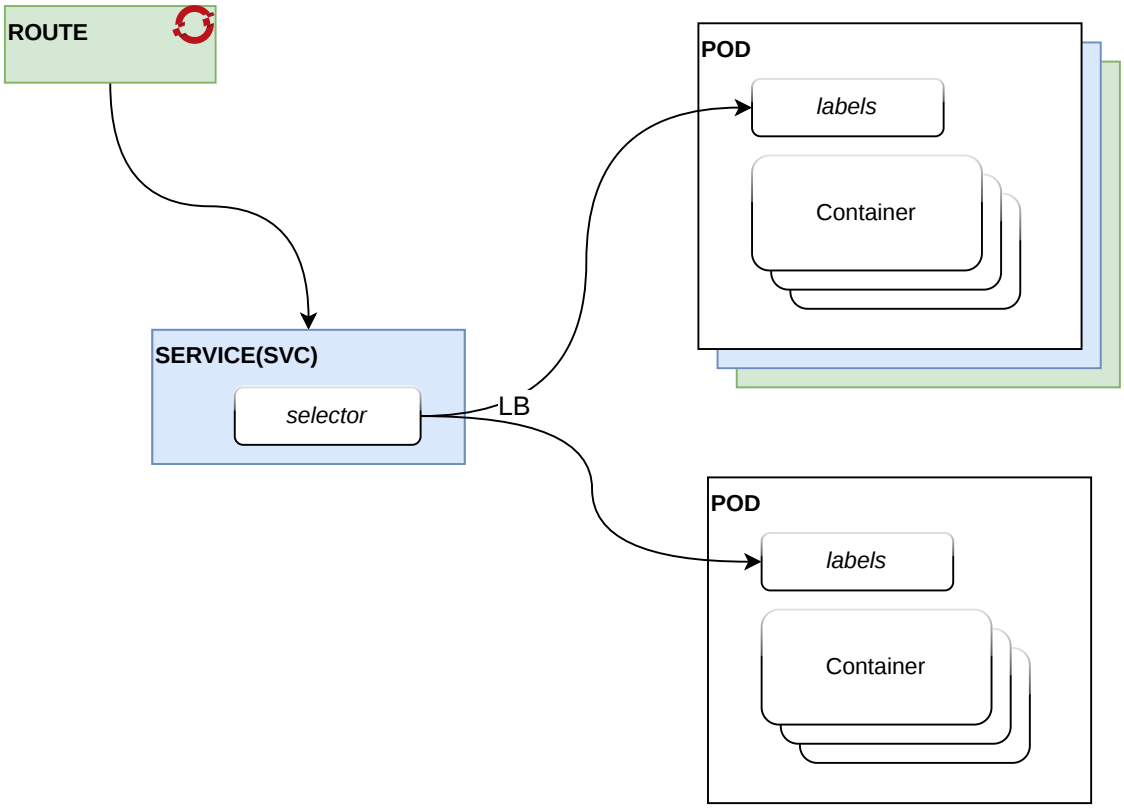
Route, Service and Pod Relationship



POD
A pod contains one or more containers.

SERVICE
A service references the pod(s) by using the label selector.
The service load balances the connections between all the pods.

ROUTE/INGRESS
A route exposes the service to the external world.

Warning: A service "can" refer to different pods, if the pods have the same label.

SERVICE(SVC)

```
oc expose <DC/DEPLOYMENT/RC/RS/POD> <RESOURCE_NAME>

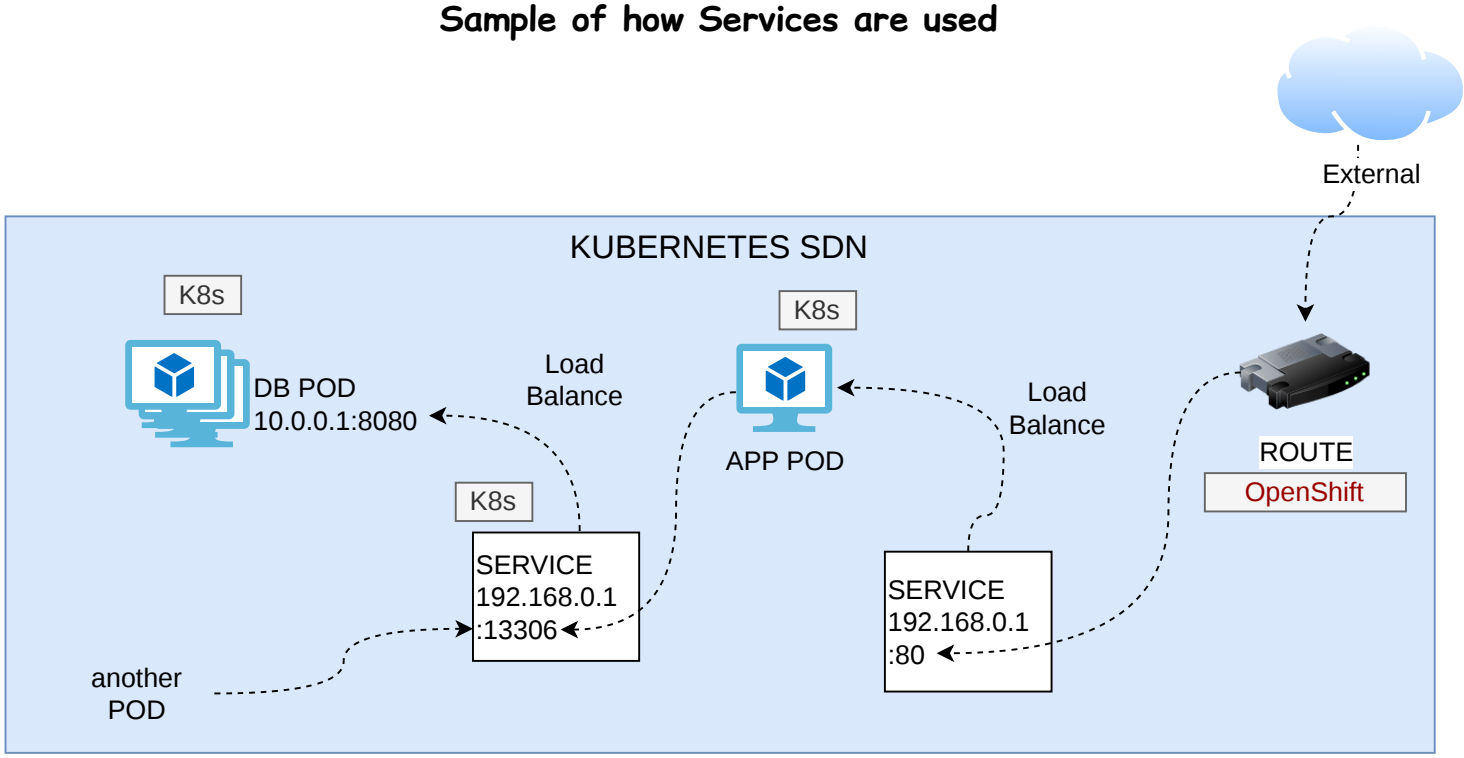
DNS NAME = <SVC>.<PROJ>[.svc.cluster.local]
ENVIRONMENT VARIABLE IN POD = <SVC>_SERVICE_HOST
```

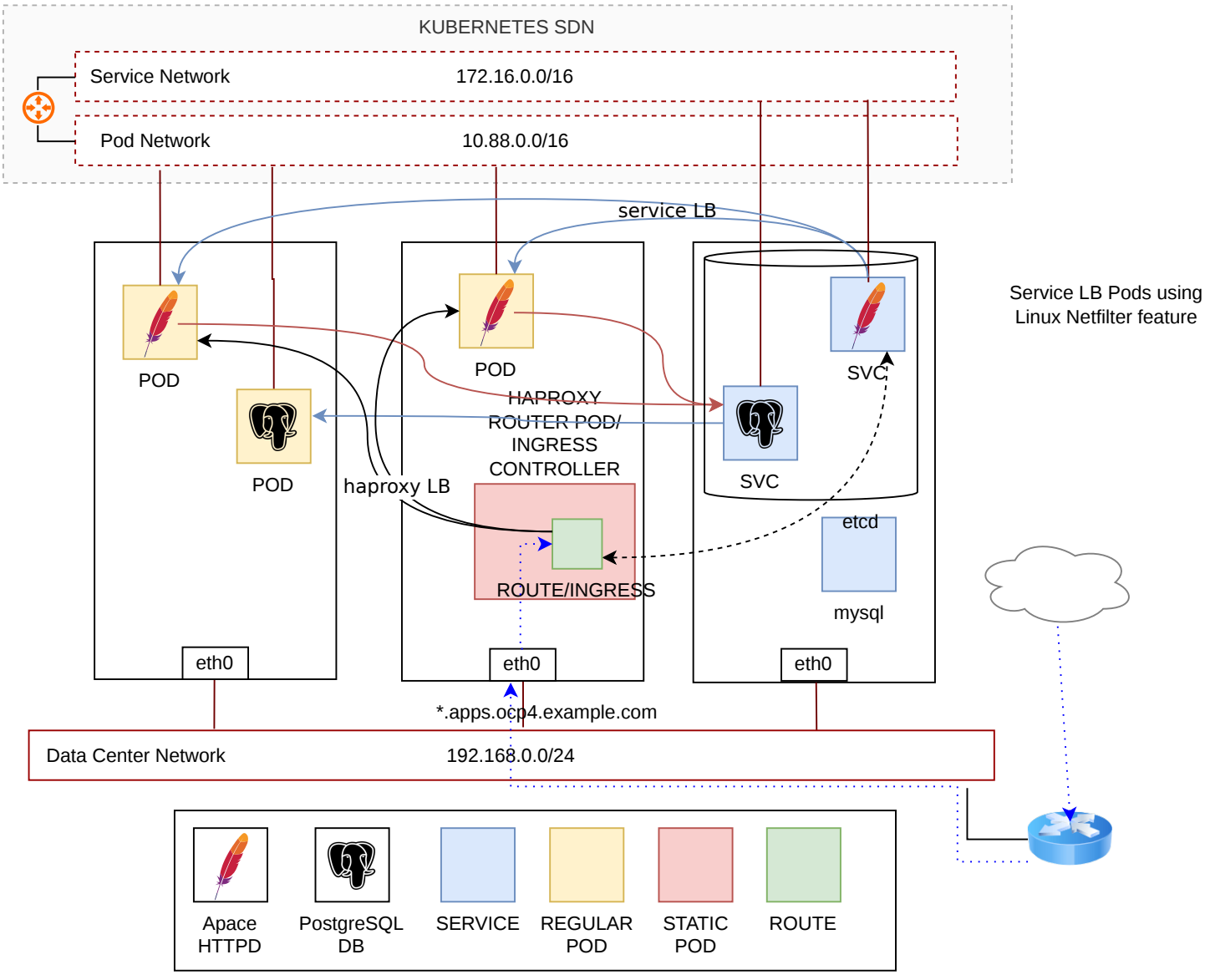
ROUTE

```
oc expose svc <SVC_NAME> [--name <ROUTE_NAME>] [--hostname <FQDN>]

DNS DEFAULT NAME = <ROUTE_NAME>-<PROJ>.<DOMAIN WILDCARD>
<DOMAIN_WILDCARD> = apps.<BASE_DOMAIN>
```

Sample of how Services are used





Template

```
apiVersion: template.openshift.io/v1

kind: Template

metadata:
  name: mytemplate
  annotations:
    description: "Description"

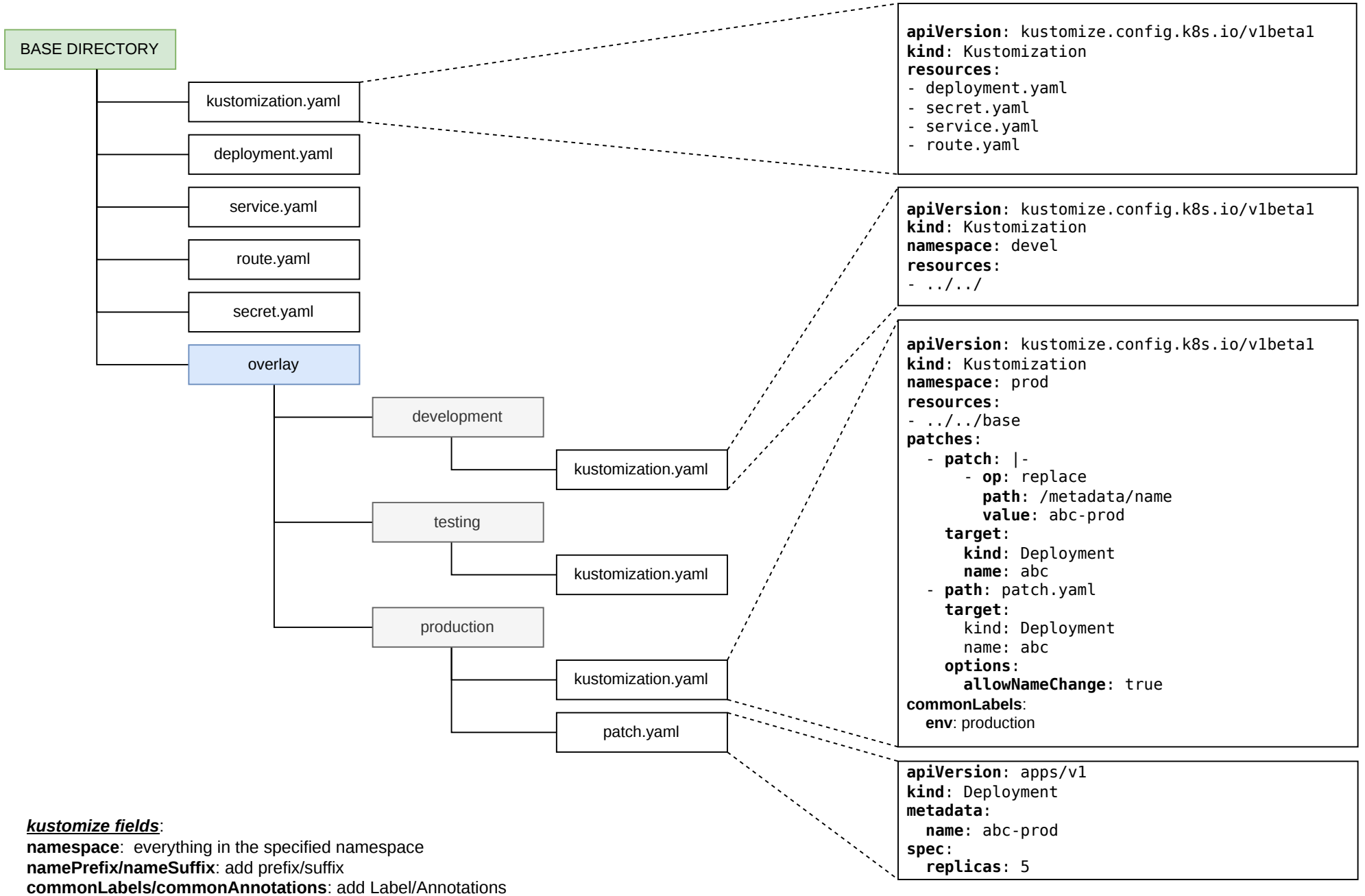
objects:
- apiVersion: v1
  kind: Pod
  metadata:
    name: ${APP_NAME}
  spec:
    containers:
    - env:
      - name: ACCESS_CODE
        value: ${APP_PASS}
      image: superapp/hyperimage
      name: myApp
      ports:
      - containerPort: 8080
        protocol: TCP

parameters:
- description: Name of Pod
  name: POD_NAME
  value: myPod
  required: true
- description: Application Secret Access Code
  name: APP_PASS
  generate: expression
  from: "[a-zA-Z0-9]{8}"

labels:
  mylabel: myapp
```



Kustomize



kustomize fields:
namespace: everything in the specified namespace
namePrefix/nameSuffix: add prefix/suffix
commonLabels/commonAnnotations: add Label/Annotations

Base Directory Overlay Directory Directory File

Commands

View manifests without applying

```
kubectl kustomize overlay/development
```

Apply/create resources

```
kubectl apply -k overlay/production
```

Delete resources

```
kubectl delete -k overlay/production
```

ConfigMap/Secret Generator
- generate CM/Secrets
- hash changes
- deployments can only detect CM *name* change

kustomize.yaml

```
kind: Kustomization
configMapGenerator:
- name: hello-app-configmap
  literals:
    - msg="Welcome!"
    - enable="true"


- name: cm-prop
  files:
    - abc.properties


- name: cm-env
  envs:
    - abc.env


secretGenerator:
- name: secret-literals
  literals:
    - ABC_USER=albert
    - ABC_PASS=einstein


- name: secret-file
  files:
    - abc.env


- name: secret-env
  envs:
    - abc.env


generatorOptions:
  disableNameSuffixHash: true
  labels:
    MYLABEL: hahaha
  annotations:
    MYNOTES: testing123
```

pod.yaml

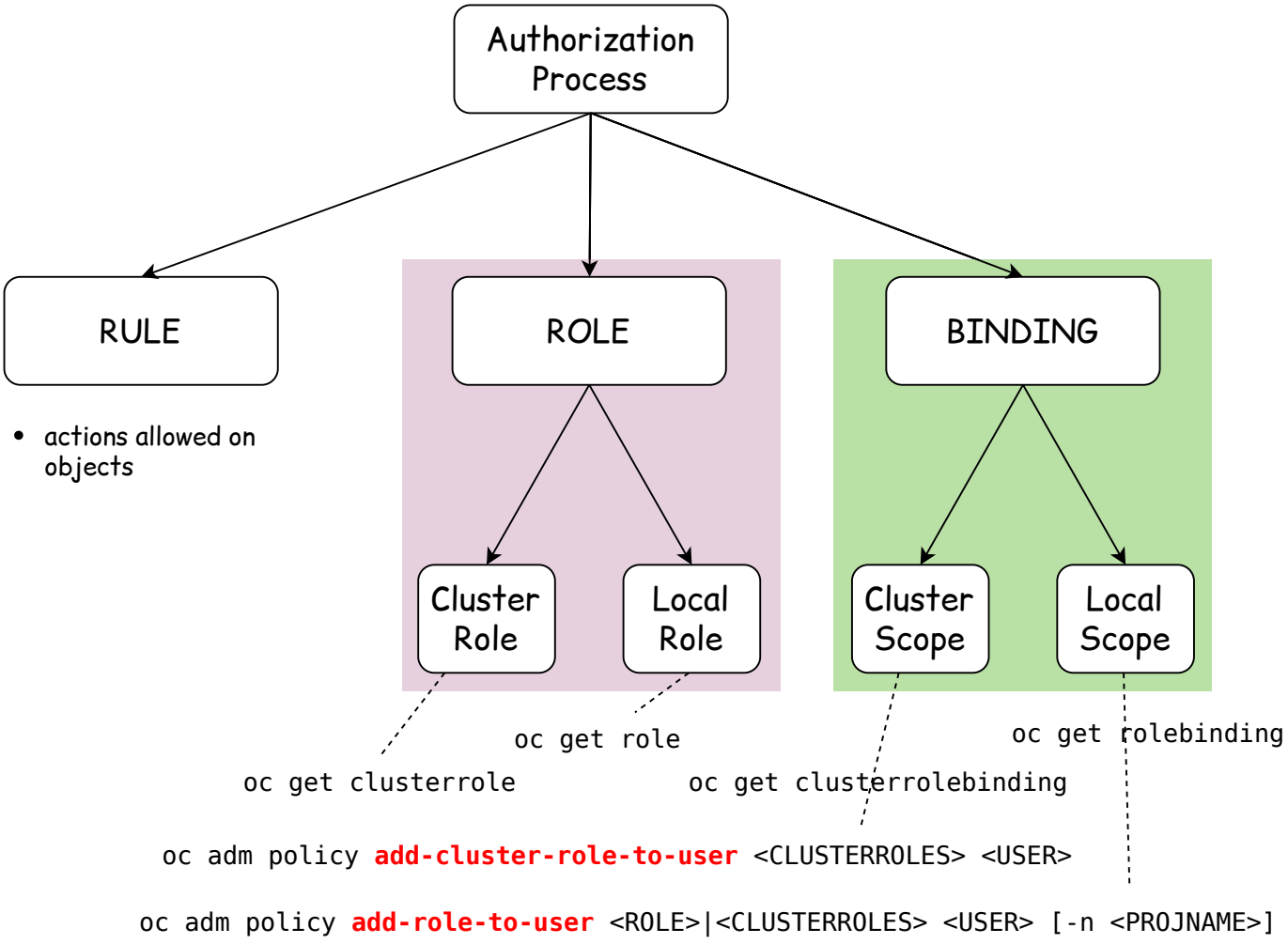
```
apiVersion: v1
kind: Pod
metadata:
  name: myPod
spec:
  containers:
  - name: hello
    image: ...
    env:
      - name: ABC_USER
        valueFrom:
          configMapKeyRef:
            name: cm-env
            key: user
```

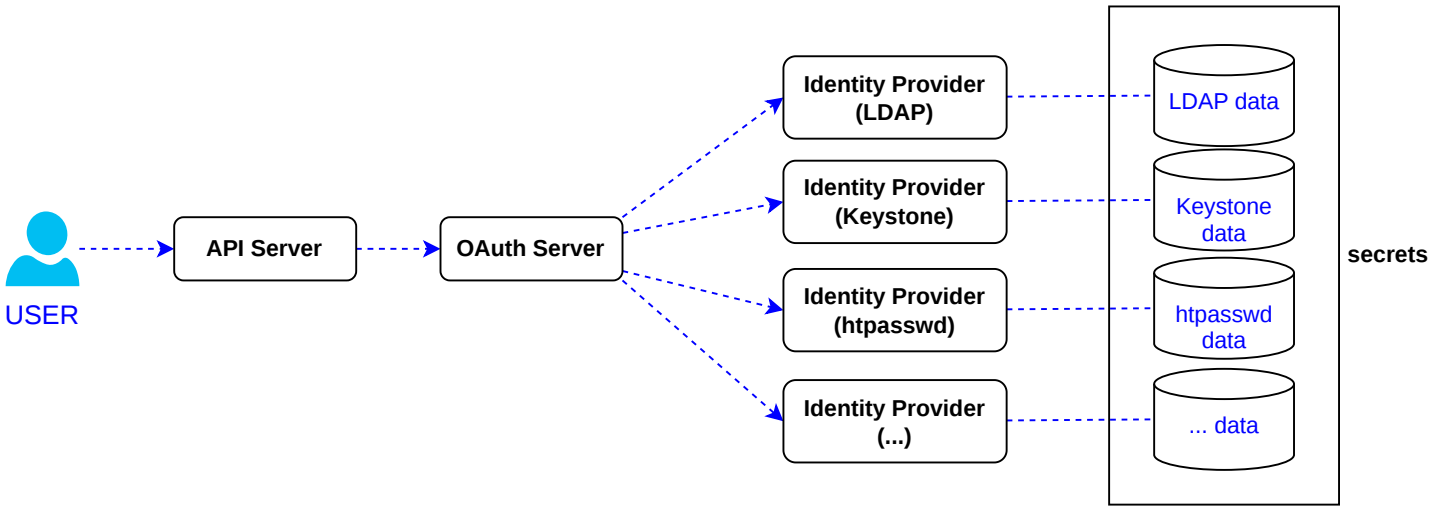
abc.env

```
user=albert
pass=einstein
```

abc.properties

```
color=blue
language=english
```


Authorization Process
RULE
ROLE
BINDING
Cluster Role
Local Role
Cluster Scope
Local Scope
actions allowed on objects
oc get role
oc get clusterrole
oc get clusterrolebinding
oc get rolebinding
oc adm policy add-cluster-role-to-user <CLUSTERROLES> <USER>
oc adm policy add-role-to-user <ROLE>|<CLUSTERROLES> <USER> [-n <PROJNAME>]




```
$ oc get co | grep auth
$ oc get pods -n openshift-authentication-operator
```

Project: *openshift-authentication-operator*

Authentication Operator — **Read** → **OAuth CR**

Manages/Configures

OAuth Server

Issues Access Tokens

Project: *openshift-authentication*

```
$ oc get pods -n openshift-authentication
```

```
$ oc get oauth cluster -o yaml
apiVersion: config.openshift.io/v1
kind: OAuth
metadata:
  name: cluster
  ...
spec:
  identityProviders:
  - htpasswd:
      fileData:
        name: htpasswd-secret
    mappingMethod: claim
    name: htpasswd_provider
    type: HTPasswd
```

secret

Project: *openshift-config*

```
$ htpasswd -cBb ./myusers <USER> <PASSWORD>
$ oc create secret generic htpasswd-secret --from-file htpasswd=./myhtpasswd -n openshift-config
```

Deploying Applications with OpenShift

Methods to create applications:

1. Using existing containerised applications

```
oc new-app --docker-image=<IMAGE>
```

2. From Source Code using S2I

```
oc new-app <URL>
```

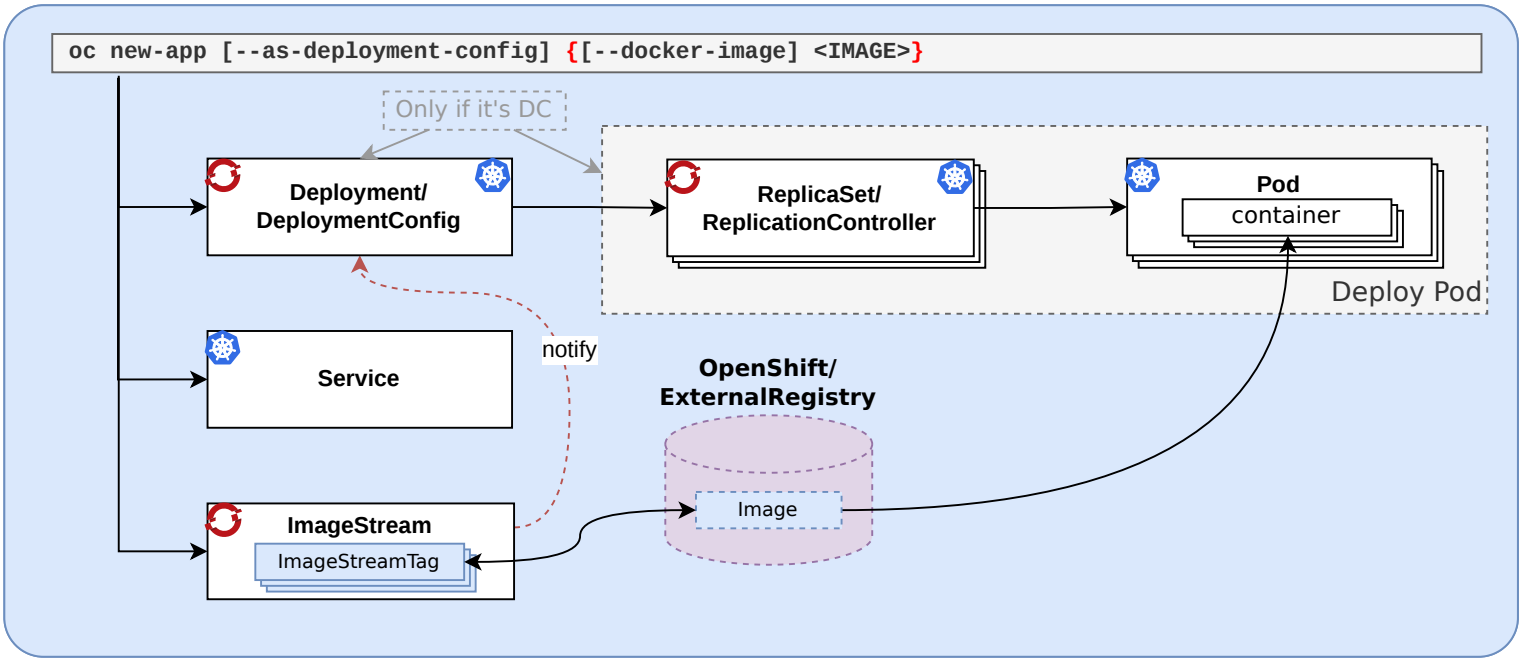
3. Using yaml/json file

```
oc new-app -f <FILE>.yaml
```

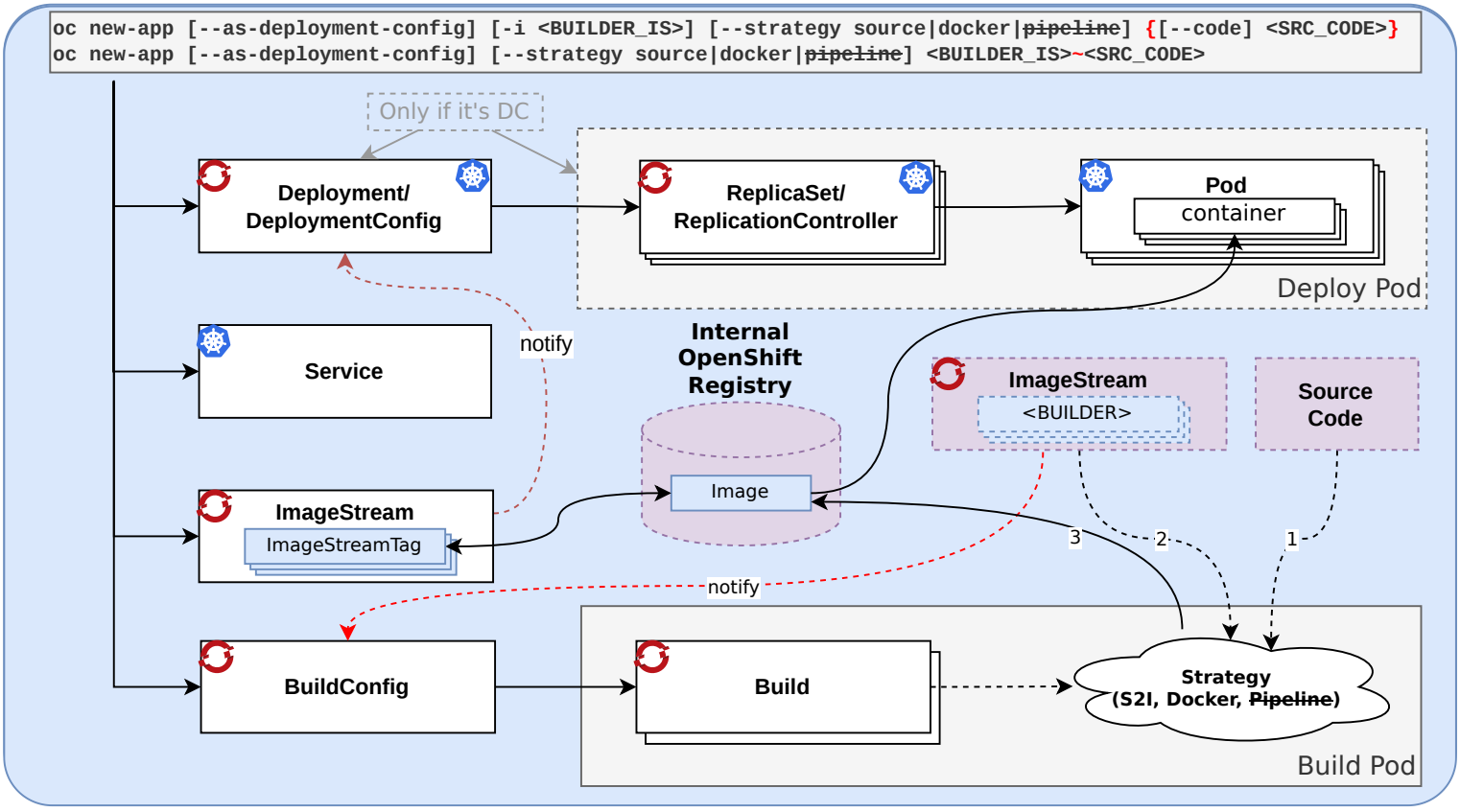
4. Using template

```
oc new-app --template=<TEMPLATE> --param=<PARAM> --param-file=<PARAM_FILE>
```

1. Use Existing Image



2. Managed Life Cycle



```
oc new-app -i myphp https://github.com/user/myapp#branch --context-dir <DIR>
```

```
oc new-app -i myphp:7.1 https://github.com/user/myapp
```

```
oc new-app myphp:7.1~https://github.com/user/myapp
```

NOTE: -i option needs git client to be installed

Options

-o json|yaml	inspect resource definitions without creating
--name <NAME>	adds a label "app=<NAME>" to all resources, Use ***`oc delete all -l "app=<NAME>"`*** to cleanup

IMPORT IMAGES

```
oc import-image <IMG_STREAM> [--confirm] --from <IMAGE> [--insecure]
where,
    <IMAGE> = <REGISTRY>[:<PORT>]/<NAMESPACE>/<REPOSITORY>[:<TAG>]
```

oc new-app command in OpenShift 4.5 makes use of *deployment* resource. Use --as-deployment-config if you wish to create *deployment config* instead.



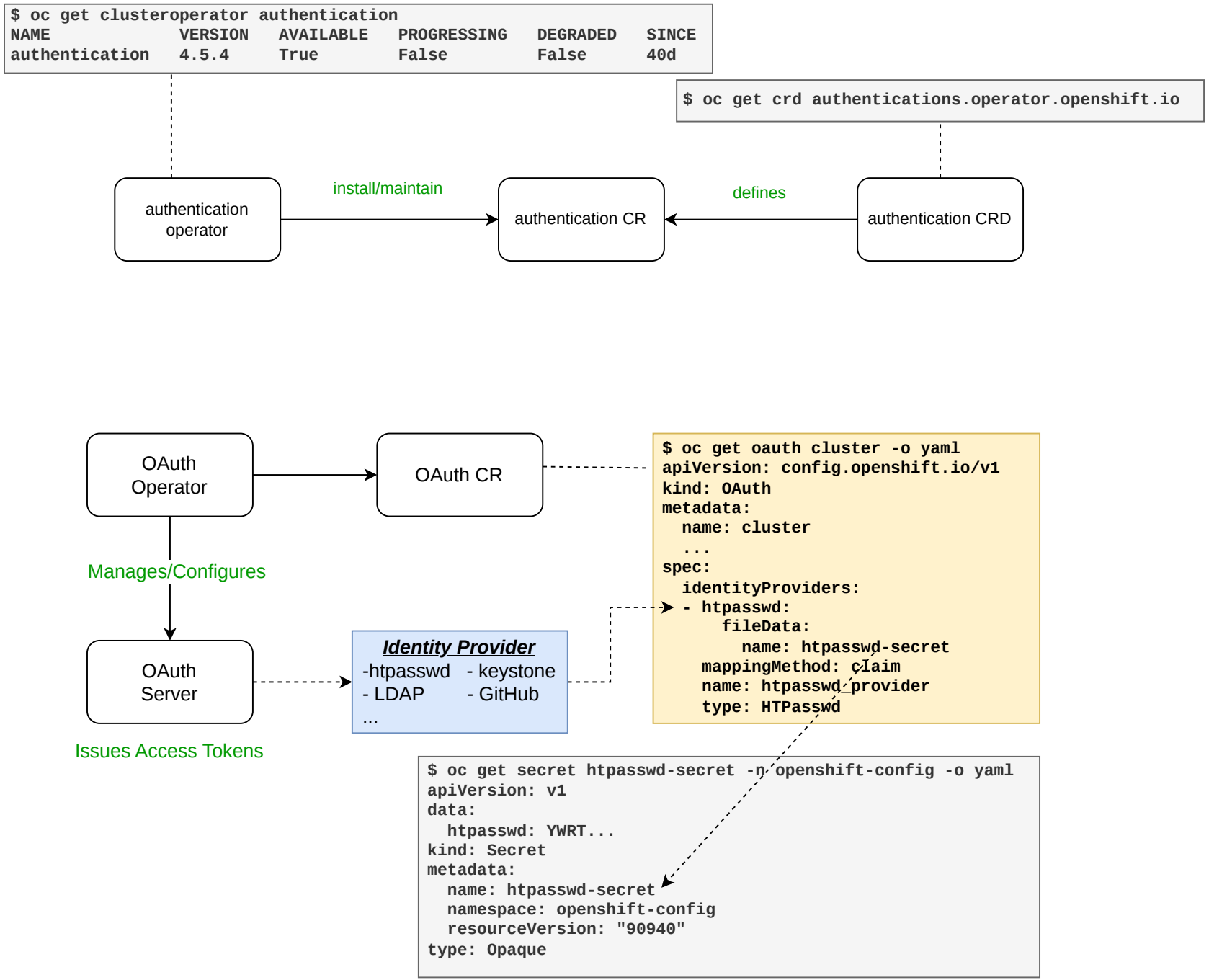
oc expose <DC/DEPLOYMENT/RC/RS/POD> <RESOURCE_NAME>

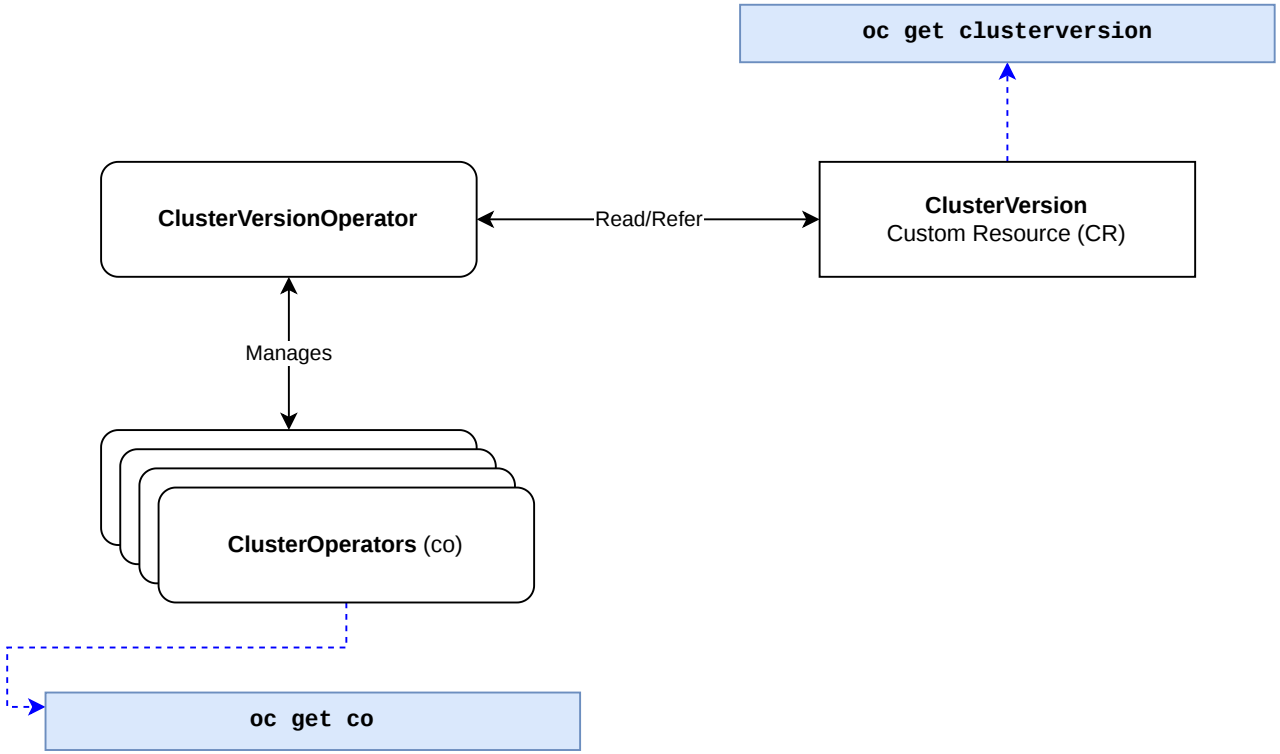
DNS NAME = <SVC>.<PROJ>[.svc.cluster.local]
ENVIRONMENT VARIABLE IN POD = <SVC>_SERVICE_HOST



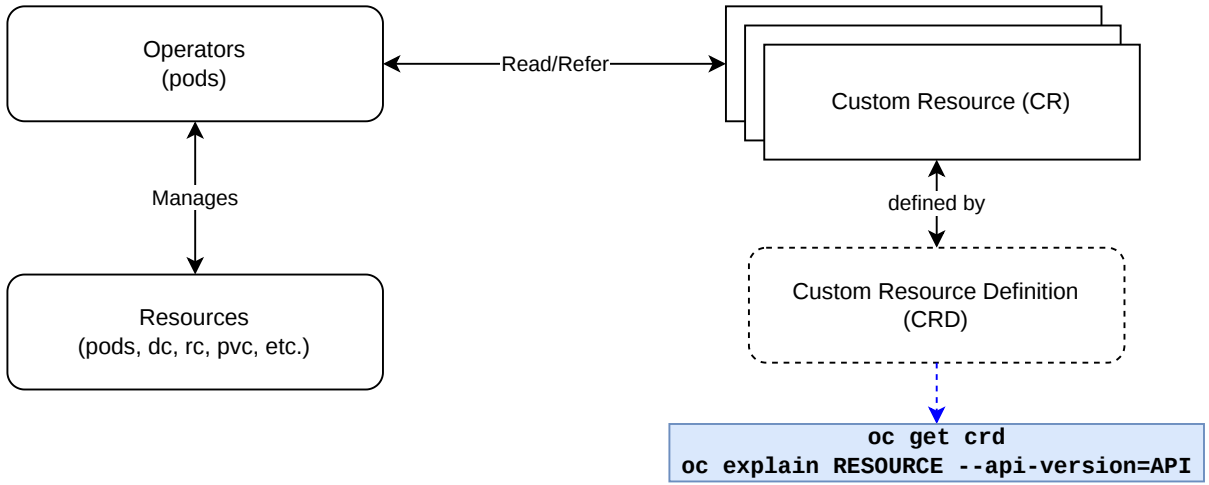
oc expose svc <SVC_NAME> [--name <ROUTE_NAME>] [--hostname <FQDN>]

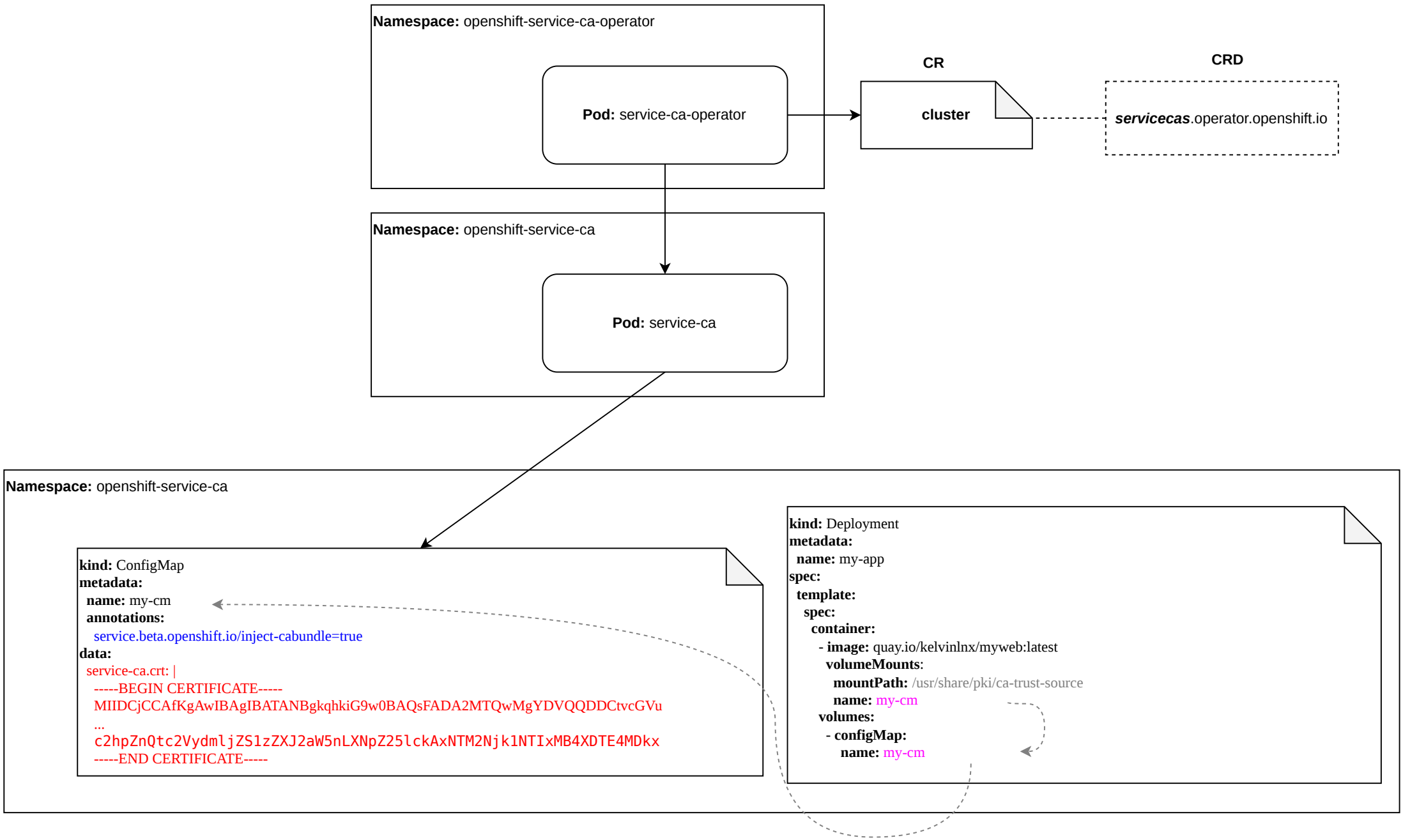
DNS DEFAULT NAME = <ROUTE_NAME>-<PROJ>.<DOMAIN WILDCARD>
<DOMAIN_WILDCARD> = apps.<BASE_DOMAIN>

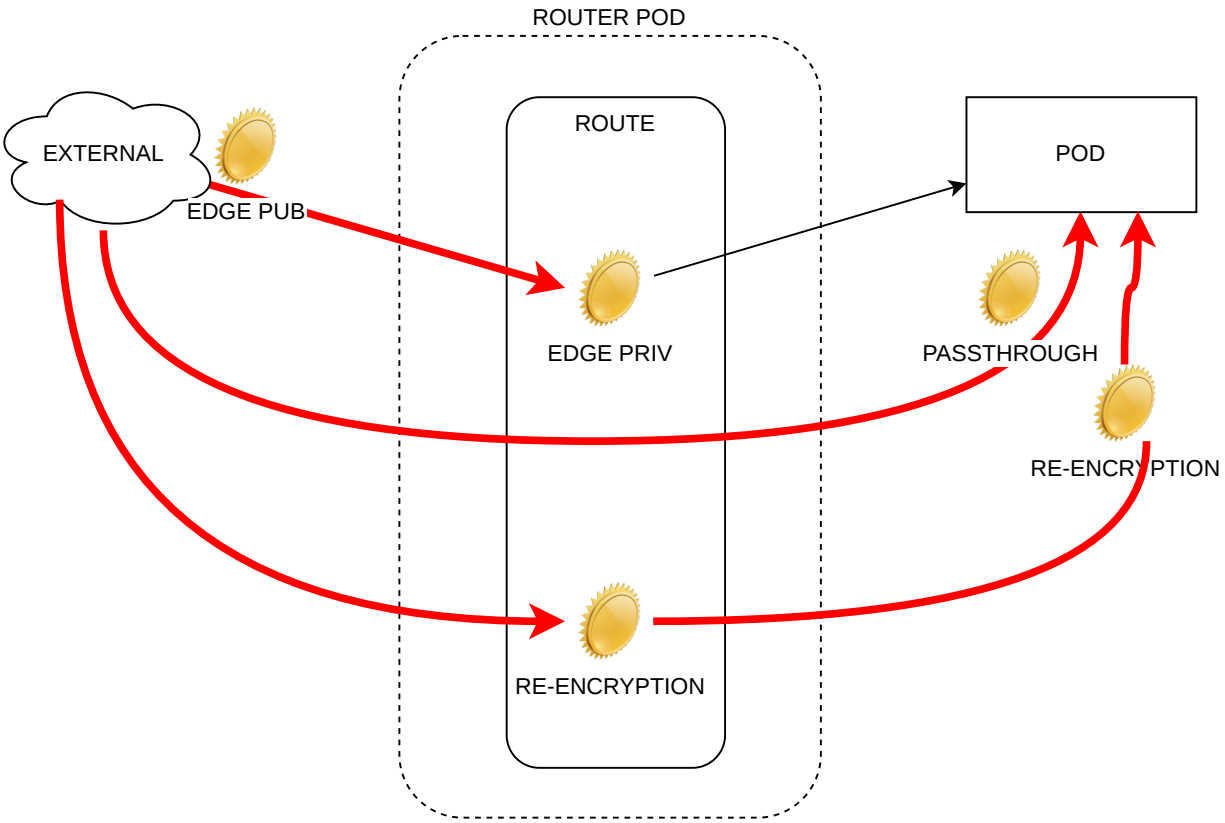

$ oc get clusteroperator authentication
NAME VERSION AVAILABLE PROGRESSING DEGRADED SINCE
authentication 4.5.4 True False False 40d
$ oc get crd authentications.operator.openshift.io
authentication operator
install/maintain
authentication CR
defines
authentication CRD
OAuth Operator
OAuth CR
$ oc get oauth cluster -o yaml
apiVersion: config.openshift.io/v1
kind: OAuth
metadata:
name: cluster
...
spec:
identityProviders:
- htpasswd:
fileData:
name: htpasswd-secret
mappingMethod: claim
name: htpasswd_provider
type: HTPasswd
Manages/Configures
OAuth Server
Identity Provider
-htpasswd - keystone
- LDAP - GitHub
...
Issues Access Tokens
$ oc get secret htpasswd-secret -n openshift-config -o yaml
apiVersion: v1
data:
htpasswd: YWRT...
kind: Secret
metadata:
name: htpasswd-secret
namespace: openshift-config
resourceVersion: "90940"
type: Opaque

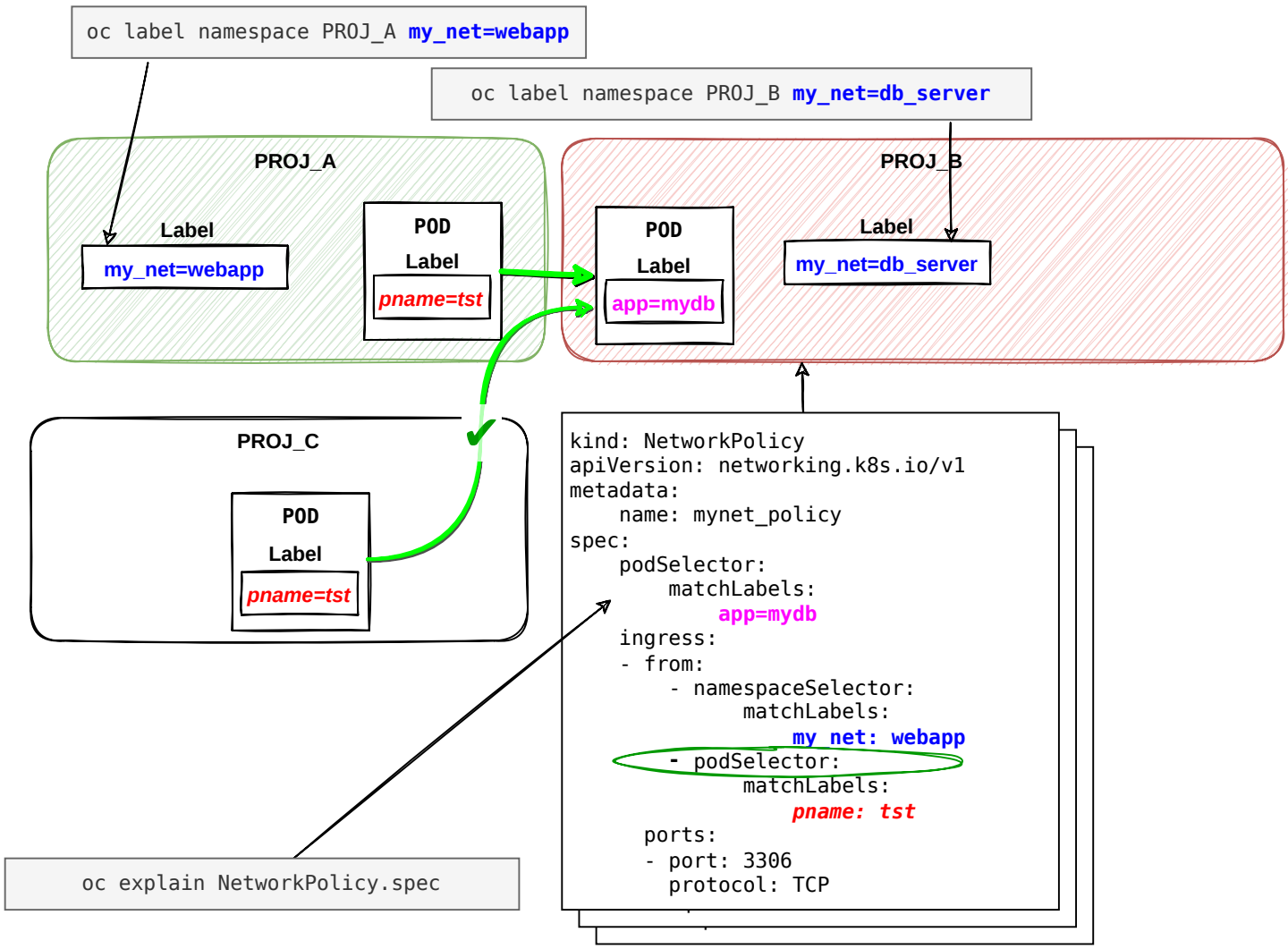


An operator is a control loop program and it can respond to events.
It communicates with the API server to manage k8s resources




Namespace: openshift-service-ca-operator
Pod: service-ca-operator
CR
cluster
CRD
servicecas.operator.openshift.io
Namespace: openshift-service-ca
Pod: service-ca
Namespace: openshift-service-ca
kind: ConfigMap
metadata:
name: my-cm
annotations:
service.beta.openshift.io/inject-cabundle=true
data:
service-ca.crt: |
-----BEGIN CERTIFICATE-----
MIIDCjCCAfKgAwIBAgIBATANBgkqhkiG9w0BAQsFADA2MTQwMgYDVQQDDCtvcGVu
...
c2hpZnQtc2VydmljZS1zZXJ2aW5nLXNpZ25lckAxNTM2Njk1NTIxMB4XDTE4MDkx
-----END CERTIFICATE-----
kind: Deployment
metadata:
name: my-app
spec:
template:
spec:
container:
- image: quay.io/kelvinlnx/myweb:latest
volumeMounts:
mountPath: /usr/share/pki/ca-trust-source
name: my-cm
volumes:
- configMap:
name: my-cm


ROUTER POD
ROUTE
EXTERNAL
EDGE PUB
POD
EDGE PRIV
PASSTHROUGH
RE-ENCRYPTION
RE-ENCRYPTION


oc label namespace PROJ_A my_net=webapp
oc label namespace PROJ_B my_net=db_server
PROJ_A
Label
my_net=webapp
POD
Label
pname=tst
PROJ_B
POD
Label
app=mydb
Label
my_net=db_server
PROJ_C
POD
Label
pname=tst
kind: NetworkPolicy
apiVersion: networking.k8s.io/v1
metadata:
name: mynet_policy
spec:
podSelector:
matchLabels:
app=mydb
ingress:
- from:
- namespaceSelector:
matchLabels:
my_net: webapp
- podSelector:
matchLabels:
pname: tst
ports:
- port: 3306
protocol: TCP
oc explain NetworkPolicy.spec